

Development of a 3D Animated Car moving Scenery Using OpenGL

course Title: Computer Graphics

Course Code: CSE-4110

Submitted By

Sabiha Akter Tanny(21CSE011)

Sadia Amrin Laaj(21CSE051)

Department of Computer Science and Engineering

University of Barishal

Submitted To

MD. Manjur Ahmed

Associate Professor

Department of Computer Science and Engineering

University of Barishal

Date of Submission: 21 June, 2026

Index

1	Abstract	3
2	Objectives	3
3	Theory	3
3.1	Geometric Modeling	3
3.2	Transformations	4
3.3	Animation	4
3.4	Viewing Transformation	4
3.5	Double Buffering	4
3.6	Procedural texture generation	4
3.7	Animation principles	4
4	Software and Hardware Requirements	5
4.1	Software	5
4.2	Hardware	5
5	Algorithm	5
6	Description of Major Functions	5
6.1	drawBox()	5
6.2	drawBall()	6
6.3	cloud()	6
6.4	mountain()	6
6.5	tree()	6
6.6	drawPineTree()	6
6.7	drawRealisticCar()	6
6.8	draw3DBoat()	6
6.9	drawMan()	6
6.10	display()	6
6.11	update()	6
6.12	Camera and Projection	7
6.13	handleSpecialKeys()	7
7	Keyboard Controls	7
8	Features	7
9	Applications	8

10 Advantages	8
11 Limitations	8
12 Individual Work Contribution	9
13 Results and Observations	9
14 Possible Improvements	11
15 Conclusion	11

1. Abstract

This experiment demonstrates the design and implementation of an interactive 3D animated scenery using OpenGL and GLUT in C++. The project creates a realistic outdoor environment containing mountains, trees, clouds, a river, a road, moving vehicles, a boat, and a human character. Different geometric primitives such as cubes, spheres, cones, triangles, and quadrilaterals are used to construct complex objects. Animation techniques are applied to simulate moving clouds, flowing river waves, and vehicle motion. User interaction is incorporated through keyboard controls that allow camera movement and object manipulation. The project illustrates fundamental concepts of computer graphics including 3D transformations, animation, lighting-like color effects, hierarchical modeling, and interactive visualization.

2. Objectives

1. The objective of this lab exercise is to design and implement a 3D computer graphics scene using the OpenGL Utility Toolkit (GLUT) in C++. The program demonstrates:
 - Procedural texture generation
 - Composition of primitive 3D shapes into complex objects (cars, boats, trees, mountains, a human figure)
 - Camera control via keyboard input
 - Frame-based animation and simple scripted vehicle/character behavior
 - A simple non-lit, color-layered approach to simulate shading (a 3D sun)

3. Theory

Computer graphics is the branch of computer science that deals with the creation, manipulation, and representation of visual images using computers. OpenGL (Open Graphics Library) is a powerful graphics API used for rendering 2D and 3D vector graphics.

3.1. Geometric Modeling

Objects are constructed using primitive shapes:

- Cubes
- Spheres
- Cones

- Triangles
- Quadrilaterals

3.2. Transformations

Translation

```
glTranslatef(x, y, z);
```

Rotation

```
glRotatef(angle, x, y, z);
```

Scaling

```
glScalef(x, y, z);
```

3.3. Animation

Animation is achieved using timer functions:

```
glutTimerFunc(16, update, 0);
```

3.4. Viewing Transformation

```
gluLookAt()
```

3.5. Double Buffering

GLUTDOUBLE is used to prevent flickering.

3.6. Procedural texture generation

Instead of loading an image file, the program synthesizes 64×64 RGB textures at runtime by assigning a base color and adding per-pixel random noise, producing a naturally varied, non-uniform appearance for grass and asphalt without needing external image assets.

3.7. Animation principles

The program uses GLUT's timer-callback mechanism (`glutTimerFunc`) to repeatedly update state variables (positions, wave phase, cloud offset) and trigger redraws, producing continuous motion at a fixed interval (16 ms = 60 FPS).

4. Software and Hardware Requirements

4.1. Software

- CodeBlocks IDE
- C++ Compiler (MinGW/GCC)
- OpenGL Library
- GLUT / FreeGLUT Library
- Windows Operating System

4.2. Hardware

- Intel/AMD Processor
- Minimum 4GB RAM
- Graphics Support
- Keyboard and Monitor

5. Algorithm

1. Initialize OpenGL and GLUT environment.
2. Enable depth testing and set background color.
3. Create helper functions for objects (cube, sphere, car, boat, etc.).
4. Construct environment (road, river, mountains, trees, sun).
5. Add animated objects (clouds, cars, waves, human movement).
6. Implement keyboard controls.
7. Use timer function for animation updates.
8. Render scene continuously.

6. Description of Major Functions

6.1. drawBox()

Creates cuboid objects used for car body, trunk, etc.

6.2. drawBall()

Creates spheres used for clouds, sun, head, etc.

6.3. cloud()

Generates cloud structures using multiple spheres.

6.4. mountain()

Creates 3D mountain using triangular faces.

6.5. tree()

Creates tree using cube trunk and sphere leaves.

6.6. drawPineTree()

Creates pine tree using cone leaves.

6.7. drawRealisticCar()

Creates detailed car model.

6.8. draw3DBoat()

Creates boat using triangles, quads, and cone.

6.9. drawMan()

Creates human figure using cube and sphere.

6.10. display()

Main rendering function.

6.11. update()

Called every 16 ms via glutTimerFunc, this function:

Oscillates the cloud X-position (cx) and advances the wave phase (wave) for the river animation. Drives a state machine for the blue car: it drives toward a stopping zone (c2z between 1.05 and 1.2), pauses, opens its door, "ejects" a pedestrian who walks away from the car over a number of frames (tracked by timer), then hides the pedestrian, closes the door, and resumes driving until it loops back to its start position. This produces a small scripted "passenger drop-off" vignette. Calls glutPostRedisplay() and re-arms the timer recursively, forming the main animation heartbeat.

6.12. Camera and Projection

`gluLookAt()` positions the camera using **camX**, **camY**, **camZ**, looking toward a fixed point ahead of the road. `reshape()` sets up a perspective projection (`gluPerspective`) that updates whenever the window is resized.

6.13. `handleSpecialKeys()`

Handles keyboard input.

7. Keyboard Controls

Key	Function
Left Arrow	Move camera left
Right Arrow	Move camera right
Up Arrow	Zoom forward
Down Arrow	Zoom backward
F2	Move red car forward
F3	Move red car backward
F6	Move camera upward
F7	Move camera downward

8. Features

- Interactive camera movement
- Animated clouds
- Dynamic river waves
- Moving cars
- Human animation
- 3D boat model
- Multiple trees and mountains
- Double buffering

9. Applications

- Computer Graphics education
- Game development
- Simulation systems
- Virtual environments
- Animation learning

10. Advantages

- Easy to understand
- Real-time rendering
- Interactive system
- Modular structure

11. Limitations

- No lighting system
- No shadows
- Limited interaction

12. Individual Work Contribution

Name	Roll	Contribution
Sabiha Akter Tanny	21CSE011	Project Idea selection and planning ,Procedural texture generation (<code>createProceduralTextures</code>); terrain, river, and curved road system (<code>getCurve</code> , road/divider rendering); Implemented river wave, environment objects: sun, clouds, mountains, pine trees; camera system and controls; overall scene assembly in <code>display()</code>
Sadia Amrin Laaj	21CSE051	Vehicle modeling (<code>drawRealisticCar</code>); Develop the car door open and closing model, boat modeling (<code>draw3DBoat</code>); pedestrian figure (<code>drawMan</code>); Moving Cloud design, animation/state-machine logic for the blue car sequence (<code>update()</code>); testing and debugging ,Developed timer-based animation logic

13. Results and Observations

The program successfully generated a realistic animated 3D roadside scenery. The environment included moving vehicles, dynamic clouds, river wave effects, mountains, trees, and a boat. Keyboard interaction allowed the user to explore the scene from different viewpoints. Smooth animation was achieved using timer-based updates and double buffering. On execution, the program opens an 800×600 window showing:

- A sky-blue background with a glowing layered sun and drifting clouds.
- A green textured terrain with a winding gray road featuring a yellow center line.
- A blue river with animated ripple lines flowing beside the road, with a small boat anchored near the bank.
- Rocky, multi-toned mountains in the background flanked and interspersed with pine trees, plus a tree-lined road edge.
- Two cars (red and blue) on the road; the blue car periodically slows, opens its door, and "drops off" a small pedestrian figure who walks a short distance before disappearing, after which the car resumes motion.
- The red car can be manually moved forward/backward with F2/F3, and the camera can be panned, zoomed, and raised/lowered with the arrow keys and F6/F7.

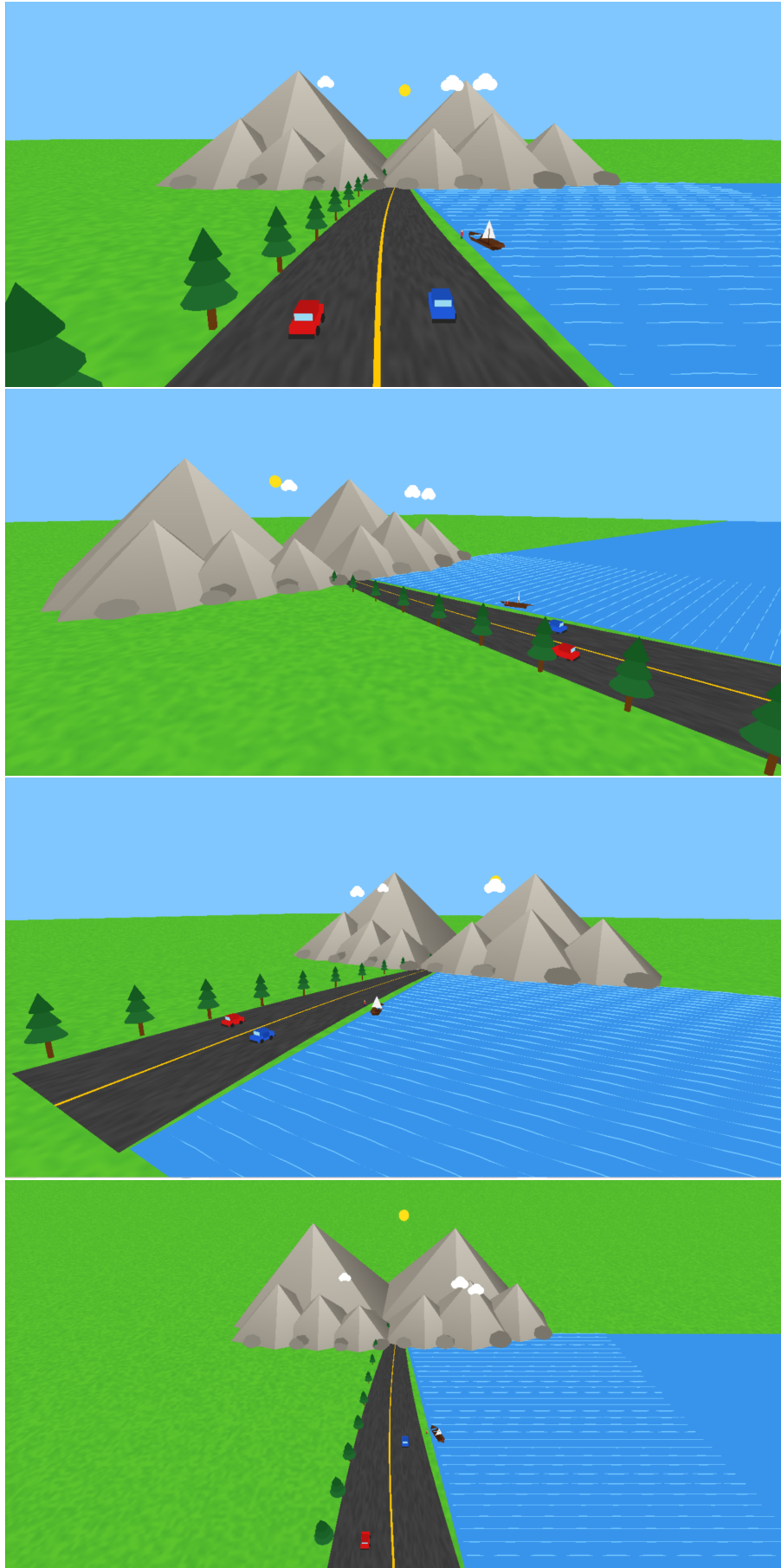


Figure 1: Output Screenshots of 3D Animated Scene (Different Views and Animations)

14. Possible Improvements

- Replace immediate-mode rendering with vertex buffer objects (VBOs) and shaders for modern OpenGL compliance and performance.
- Add real lighting (`GL_LIGHTING`) with normals for more physically accurate shading instead of manually painted triangle colors.
- Load external texture image files (e.g., via `stb_image` or `SOIL`) instead of procedurally generated noise textures for higher visual fidelity.
- Parameterize vehicle/pedestrian animation with delta-time rather than a fixed 16 ms timer for frame-rate independence.

15. Conclusion

This program is a practical demonstration of building a moderately complex, animated 3D scene entirely from OpenGL's basic primitives and the fixed-function pipeline, without relying on external 3D models, image files, or a lighting system. It illustrates several core graphics concepts in combination: procedural texture synthesis, hierarchical transformations for composite object modeling, manual triangle-color shading to simulate depth and lighting, parametric curves for road/terrain layout, and timer-driven animation with simple scripted behavior (state-machine logic for the vehicle/pedestrian sequence). The exercise reinforces how a visually rich scene can be assembled from simple geometric building blocks (cubes, spheres, cones, and triangles) governed by careful use of the matrix stack.